

快手前端研发工程师面试 (agent方向)

一面（技术基础） · 二面（深入原理与项目） 三面（架构与综合能力）

一面（技术基础）

1. 简单介绍下你对闭包的理解，以及它在实际开发中会有哪些副作用？
2. 说一下事件循环（Event Loop）的机制，微任务和宏任务的执行顺序是怎样的？
3. 谈谈 CSS 中的 BFC，它解决了什么问题？
4. React 中key 的作用是什么？如果用 index 作为 key 会有什么问题？
5. 在 Agent 交互场景中，经常用到 SSE（Server-Sent Events），它和 WebSocket 有什么区别？
6. 手写实现一个 Promise.all 方法。
7. 数组转树形结构：给定一个包含id，parentId 的扁平数组，如何高效转换成树？
8. 谈谈浏览器的强缓存和协商缓存。
9. TypeScript 中的 interface 和 type 有什么区别？
10. 算法题：给定一个只包括 '<code>'</code>'、'('、')'、',' 、'{'、'}'、'['、']' 的字符串，判断字符串是否有效。
11. 什么是虚拟列表（Virtual List）？它的实现原理是什么？
12. 谈谈 JS 中的深拷贝和浅拷贝，如何实现一个深拷贝？

二面（深入原理与项目）

1. React Fiber 架构解决了什么问题？它的工作原理是怎样的？
2. 在开发 Agent 产品时，如何实现一个流畅的“打字机”输出效果？
3. 如何处理 Agent 返回的长文本中的 Markdown 和 LaTeX 公式渲染？
4. 你们项目里是如何进行前端性能优化的？有哪些量化指标？
5. Webpack 的构建流程是怎样的？你做过哪些打包优化？
6. 谈谈微前端架构，为什么需要它？常用的方案有哪些？
7. 在 Agent 这种复杂状态的应用中，你会选择哪种状态管理方案？为什么？
8. 快手内部有很多跨端场景，你对跨端方案（如小程序、KwaiNative）有了解吗？
9. 前端如何捕获并处理异常？对于白屏异常怎么监控？

10. 手写题：实现一个带并发限制的异步调度器 Scheduler。
11. 谈谈 Canvas 和 SVG 的区别，在 Agent 的工作流（Workflow）画布中你会选哪个？
12. 什么是 Service Worker？它能为 Agent 应用带来什么提升？

三面（架构与综合能力）

1. 如果让你从零设计一个 Agent 智能体平台的前端架构，你会考虑哪些核心模块？
2. 在 Agent 项目中，前端如何参与 Prompt Engineering（提示词工程）的优化？

一面（技术基础）

1. 简单介绍下你对闭包的理解，以及它在实际开发中会有哪些副作用？

难度：大

考点：JavaScript 作用域、内存管理答案要点：

闭包是指一个函数可以访问其外部作用域中的变量，即使外部函数已经执行完毕。

- 形成原因：函数嵌套函数，内部函数引用了外部函数的变量，导致变量无法被销毁。
- 常见场景：封装私有变量、柯里化（Currying）、防抖节流函数。
- 内存影响：如果闭包长期存在且引用了大量对象，会导致内存无法被垃圾回收机制（GC）释放。
- 解决方法：在不需要使用时将引用置为 null，解除引用关系。

常见坑：

- 认为闭包一定会导致内存泄漏。
- 忽略了循环中使用 var 定义变量导致的闭包陷阱。

追问：

- 浏览器是如何判断一个变量是否应该被回收的？

2. 说一下事件循环（Event Loop）的机制，微任务和宏任务的执行顺序是怎样的？

难度：太大

考点：JS 异步执行机制

答案要点：

事件循环是 JS 处理异步任务的核心机制，它通过任务队列来协调主线程的执行。

- 执行流程：先执行同步代码（调用栈），栈空后检查微任务队列。
- M17\$ (Microtask) : Promise.then, MutationObserver, process.nextTick.

- 宏任务 (Macrotask) : `setTimeout`、`setInterval`、I/O、UI 渲染。
- 循环规律：每执行完一个宏任务，都会清空当前的微任务队列，然后再进行 UI 渲染。

常见坑：

- 混淆 Promise 构造函数（同步）和 `.then`（异步）。
- 不清楚 `await` 后面的代码会被包装成微任务。

2. 说一下事件循环 (Event Loop) 的机制，微任务和宏任务的执行顺序是怎样的？

- 难度：中等
- 考点：JS 异步运行机制、执行栈
- 答案要点：
 - 核心机制：JS 是单线程的，同步代码在主线程（执行栈）执行。异步任务被分为宏任务 (MacroTask) 和微任务 (MicroTask)。
 - 执行顺序：
 - i. 执行主线程中的同步代码。
 - ii. 清空当前的微任务队列（所有微任务依次执行）。
 - iii. 尝试进行 UI 渲染（如果需要）。
 - iv. 执行一个宏任务。
 - v. 回到第2步，循环往复。
 - 分类：
 - 微任务：`Promise.then/catch/finally`、`MutationObserver`、`process.nextTick` (Node.js)。
 - 宏任务：`setTimeout`、`setInterval`、`setImmediate` (Node.js)、I/O、UI Rendering、`MessageChannel`。
- 图解：
- 追问：
 - `requestAnimationFrame` 属于宏任务还是微任务？（通常在渲染之前执行，不完全属于这两者，但在浏览器循环中位置固定）。
 - Node.js 的 Event Loop 和浏览器有什么区别？

3. 谈谈 CSS 中的 BFC，它解决了什么问题？

- **难度：** 中等
 - **考点：** CSS 布局原理
 - **答案要点：**
 - **定义：** BFC (Block Formatting Context, 块级格式化上下文) 是一个独立的渲染区域，内部元素的渲染不会影响到外部。
 - **触发条件：**
 - `overflow` 不为 `visible` (如 `hidden`, `auto`)。
 - `float` 不为 `none`。
 - `position` 为 `absolute` 或 `fixed`。
 - `display` 为 `inline-block`, `flex`, `grid`, `flow-root` 等。
 - **解决的问题：**
 - i. **清除浮动：** 计算 BFC 高度时，浮动元素也参与计算（解决父元素高度塌陷）。
 - ii. **防止 Margin 重叠：** 属于不同 BFC 的两个相邻元素，垂直 Margin 不会重叠。
 - iii. **自适应布局：** BFC 区域不会与浮动元素重叠（常用于两栏布局）。
 - **追问：**
 - 最新的 CSS 标准中，推荐用什么属性专门创建 BFC？（`display: flow-root`）。
-

4. React 中 key 的作用是什么？如果用 index 作为 key 会有什么问题？

- **难度：** 中等
 - **考点：** Virtual DOM、Diff 算法
 - **答案要点：**
 - **作用：** Key 是虚拟 DOM 对象的唯一标识。在 Diff 算法中，React 依靠 Key 来判断节点是**被复用、修改、还是删除/新增**，从而提高更新效率。
 - **Index 作为 Key 的问题：**
 - i. **性能损耗：** 如果列表顺序发生变化（如在头部插入），所有节点的 index 都会变，React 会认为所有组件都变了，导致不必要的全量更新。
 - ii. **状态错乱：** 如果列表项中有非受控组件（如 input 输入框），复用节点时会保留旧的 DOM 状态，导致输入框内容对应错误。
 - **追问：**
 - 为什么不建议用 `Math.random()` 做 key？（每次 Render 都会生成新值，导致 DOM 无法复用，强制重新挂载，极度影响性能）。
-

5. 在 Agent 交互场景中，经常用到 SSE（Server-Sent Events），它和 WebSocket 有什么区别？

• 难度：中等

• 考点：HTTP 通信、实时传输协议

• 答案要点：

- **SSE (Server-Sent Events):**

- **单向通信：**Server -> Client。
- **协议：**基于 HTTP 协议。
- **特性：**支持断线自动重连，传输文本（Text），轻量级。
- **场景：**ChatGPT 流式回复、股票行情推送、日志监控。

- **WebSocket:**

- **双向通信：**全双工（Full Duplex）。
- **协议：**独立的 TCP 协议（握手时利用 HTTP Upgrade）。
- **特性：**支持二进制和文本，心跳检测机制通常需要自己实现。
- **场景：**即时通讯（IM）、多人在线游戏。

• 追问：

- SSE 如何通过 POST 请求实现？（标准 EventSource API 只支持 GET，若需 POST 需使用 `fetch` 配合 `ReadableStream` 模拟）。

6. 手写实现一个 Promise.all 方法

难度：中等

考点：异步并发控制

答案要点：

Promise.all 接收一个 Promise 数组，当所有 Promise 都成功时返回结果数组，只要有一个失败就立即拒绝。

- 计数器：需要一个变量记录已完成的 Promise 数量。
- 结果存储：创建一个数组，按输入顺序存储每个 Promise 的 resolve 结果。
- 状态处理：遍历输入数组，对每个项调用 Promise.resolve，并在 then 中累加计数。
- 边界条件：处理空数组的情况，应立即返回一个 resolve（[]）。

```

1  function myPromiseAll(promises) {
2      return new Promise((resolve, reject) => {
3          if (!Array.isArray(promises)) {
4              return reject(new TypeError("Argument must be an array"));
5          }
6
7          let results = [];
8          let count = 0;
9          const len = promises.length;
10
11         if (len === 0) {
12             return resolve([]);
13         }
14
15         promises.forEach((p, index) => {
16             // 使用 Promise.resolve 兼容非 Promise 值
17             Promise.resolve(p).then(
18                 (value) => {
19                     results[index] = value; // 保证结果顺序和输入顺序一致
20                     count++;
21                     if (count === len) {
22                         resolve(results);
23                     }
24                 },
25                 (reason) => {
26                     reject(reason); // 只要有一个失败，整体立即失败
27                 }
28             );
29         });
30     });
31 }

```

- 追问:

- `Promise.all` 和 `Promise.allSettled` 有什么区别?

7. 数组转树形结构

- 难度: 中等

- 考点: 数据结构转换、算法复杂度

- 答案要点:

- 思路: 利用 Map 对象 (或对象哈希) 存储所有节点引用, 通过一次遍历将节点挂载到其 `parentId` 对应的 `children` 数组中。时间复杂度为 $O(n)$ 。

- 代码实现:

代码块

```
1  function arrayToTree(items) {
2      const result = []; // 存放根节点const itemMap = {}; // 存放 id -> 节点 的映射
    // 1. 先把所有项转为 Map 存储, 并初始化 childrenfor (const item of items) {
3          itemMap[item.id] = { ...item, children: [] };
4      }
5
6      // 2. 再次遍历, 组装树for (const item of items) {
7          const id = item.id;
8          const pid = item.parentId;
9          const treeItem = itemMap[id];
10
11         if (pid === 0 || !itemMap[pid]) {
12             // 根节点 (假设0是根或找不到父节点)
13             result.push(treeItem);
14         } else {
15             // 非根节点, 挂载到父节点下
16             itemMap[pid].children.push(treeItem);
17         }
18     }
19     return result;
20 }
```

8. 谈谈浏览器的强缓存和协商缓存

- 难度：中等
- 考点：HTTP 缓存机制
- 答案要点：
 - 强缓存（不需要请求服务器）：
 - 浏览器判断本地缓存未过期，直接读取，状态码 200 (from memory/disk cache)。
 - 字段： `Cache-Control: max-age=3600` (优先级高，HTTP/1.1)， `Expires` (HTTP/1.0)。
 - 协商缓存（需要请求服务器）：
 - 强缓存失效后，浏览器携带缓存标识向服务器发起请求，服务器判断文件是否更新。
 - 未更新：返回 304 Not Modified，继续使用本地缓存。
 - 已更新：返回 200 和新内容。

- 字段: ETag / If-None-Match (优先级高, 基于内容哈希), Last-Modified / If-Modified-Since (基于时间)。

- 追问:

- 用户在浏览器点击“刷新”按钮和地址栏回车, 缓存行为有什么区别?

9. TypeScript 中的 interface 和 type 有什么区别?

- 难度: 简单

- 考点: TS 基础

- 答案要点:

- **相同点:** 都可以描述对象或函数签名, 都支持扩展 (extends 或 &)。
- **不同点:**
 - i. **Interface:** 支持**声明合并** (Declaration Merging), 即定义同名 interface 会自动合并属性 (适合写第三方库)。更侧重于面向对象接口。
 - ii. **Type:** 支持**基本类型别名**、**联合类型** (Union Types |)、**元组**。不能重复声明。
- **最佳实践:** 编写库或定义对象形状时优先用 Interface, 定义复杂关系 (如 Utility Types) 或联合类型时用 Type。

10. 算法题: 有效的括号

- 难度: 简单

- 考点: 栈 (Stack) 数据结构

- 思路:

- a. 创建一个栈。
- b. 遍历字符串, 遇到左括号入栈。
- c. 遇到右括号, 检查栈顶元素是否匹配对应的左括号。若匹配则出栈, 不匹配则无效。
- d. 遍历结束, 若栈为空则有效, 否则无效。

- 代码实现:

JavaScript

代码块

```
1 function isValid(s) {  
2   const stack = [];  
3   const map = { ')': '(', ']': '[', '}': '{' };
```



```
4
5   for (let char of s) {
6       if (!map[char]) {
7           // 是左括号，入栈
8           stack.push(char);
9       } else {
10          // 是右括号，判断栈顶if (stack.pop() !== map[char]) return false;
11      }
12  }
13  return stack.length === 0;
14 }
```

11. 什么是虚拟列表（Virtual List）？它的实现原理是什么？

- **难度：**中等/困难
- **考点：**高性能渲染、大数据量优化
- **答案要点：**
 - **定义：**一种长列表优化技术，只渲染**可视区域（Viewport）**内的元素，不可见区域不渲染DOM。
 - **实现原理：**
 - i. **容器：**设置一个固定高度的容器，`overflow-y: auto`。
 - ii. **撑开高度：**内部放一个空 `div`，高度设为 `数据总数 * 每项高度`，让滚动条呈现真实长度。
 - iii. **计算渲染范围：**监听 `scroll` 事件，根据 `scrollTop` 计算出当前应该显示的 `startIndex` 和 `endIndex`。
 - iv. **偏移：**将渲染出来的列表项列表绝对定位或使用 `transform: translateY` 偏移到正确的位置，使其刚好处于视口中。
- **图解：**
- **追问：**
 - 如果列表项高度不固定（动态高度），怎么实现？（需要预估高度 + 渲染后缓存真实高度 + 滚动位置修正）。

12. 谈谈 JS 中的深拷贝和浅拷贝，如何实现一个深拷贝？

- **难度：**中等
- **考点：**数据类型、递归、内存引用

答案要点：

- **浅拷贝**：只拷贝对象的第一层属性，如果属性是引用类型，拷贝的是内存地址（如 `Object.assign`，`...` 扩展运算符）。
- **深拷贝**：递归拷贝所有层级，新旧对象互不影响。
- **简单实现**：`JSON.parse(JSON.stringify(obj))`。缺点：无法处理函数、`undefined`、`Symbol`、`Date`（变字符串）、循环引用。
- **手写 DeepClone 要点**：
 - i. 判断类型，如果是基本类型直接返回。
 - ii. 处理数组和对象。
 - iii. **关键**：使用 `WeakMap` 记录已拷贝的对象，解决**循环引用**问题。
 - iv. （进阶）处理 `Date`, `RegExp`, `Map`, `Set` 等特殊类型。

JavaScript

代码块

```
1  function deepClone(obj, hash = new WeakMap()) {
2    if (obj === null || typeof obj !== 'object') return obj;
3    if (obj instanceof Date) return new Date(obj);
4    if (obj instanceof RegExp) return new RegExp(obj);
5
6    // 查表，解决循环引用
7    if (hash.has(obj)) return hash.get(obj);
8
9    // 获取构造函数，保留原型链，同时支持 Array 和 Object
10   const cloneObj = new obj.constructor();
11   hash.set(obj, cloneObj);
12
13   for (let key in obj) {
14     if (obj.hasOwnProperty(key)) {
15       cloneObj[key] = deepClone(obj[key], hash);
16     }
17   }
18   return cloneObj;
19 }
```

二面（深入原理与项目）

1. React Fiber 架构解决了什么问题？它的工作原理是怎样的？

难度：大

考点：React 核心原理

答案要点：

Fiber 是 React 16 引入的一种新的协调引擎，主要为了解决大型应用在更新时的掉帧卡顿问题。

- 核心问题：旧日的 Stack Reconciler 是递归同步执行的，一旦开始无法中断，会阻塞主线程。
- 任务拆分：Fiber 将更新过程拆分成多个微小的任务单元。
- 可中断与恢复：利用 requestIdleCallback 机制，在浏览器空闲时执行任务，如果有高优先级任务（如输入）则中断。
- 双缓存技术：在内存中构建一棵 workInProgress 树，完成后一次性提交到真实 DOM。

常见坑：

- 认为 Fiber 提高了计算速度（实际上增加了计算量，但提高了响应性）。

追问：

- Fiber 节点上的 effectTag 是做什么用的？

2. 在开发 Agent 产品时，如何实现一个流畅的“打字机”输出效果？

- 核心技术：前端通过 SSE (Server-Sent Events) 接收流式数据。
- 优化细节：
 - a. 缓冲队列：不要收到一个字符就 `setState` 一次（会导致频繁重排重绘）。建立一个 Buffer 队列，每隔固定时间（如 50ms）或积累一定字符后，批量更新 UI。
 - b. 光标闪烁：使用 CSS 伪元素 `::after` 实现光标，流式传输结束时移除。
 - c. 自动滚动：判断用户当前是否在底部。如果在，跟随内容自动滚动；如果用户手动向上滚动查看历史，则暂停自动滚动，避免干扰。

3. 如何处理 Agent 返回的长文本中的 Markdown 和 LaTeX 公式渲染？

- 技术选型：
 - Markdown: `react-markdown` 或 `markdown-it`。
 - LaTeX: `KaTeX` (性能优于 MathJax) 或 `react-katex`。
 - 代码高亮: `prismjs` 或 `highlight.js`。
- 难点处理：
 - a. 混合渲染：编写自定义插件（Plugin），在 Markdown 解析流中识别 `$` 或 `$$` 符号，交给 KaTeX 渲染。
 - b. 流式截断问题：在 SSE 传输中，Markdown 语法（如代码块 `````）可能只传了一半，导致渲染崩坏。需要编写修正逻辑，检测未闭合的标签并在渲染前临时补全。

- c. **安全性**：必须使用 `DOMPurify` 过滤 HTML，防止 XSS 攻击（Prompt 注入攻击可能导致 LLM 输出恶意脚本）。

4. 你们项目里是如何进行前端性能优化的？有哪些量化指标？

- **量化指标 (Core Web Vitals)**:
 - **LCP** (最大内容渲染时间)：衡量加载速度。
 - **CLS** (累积布局偏移)：衡量视觉稳定性。
 - **FID/INP** (首次输入延迟)：衡量交互响应度。
- **优化手段**:
 - a. **构建层面**：Webpack/Vite 分包 (SplitChunks)、Tree Shaking、Gzip/Brotli 压缩。
 - b. **网络层面**：CDN 加速、HTTP/2、图片 WebP 格式化、DNS 预解析。
 - c. **代码层面**：路由懒加载 (Lazy Load)、组件按需加载、虚拟列表 (Virtual List)、React.memo 减少重渲染。

5. Webpack 的构建流程是怎样的？你做过哪些打包优化？

- **构建流程**:
 - a. **Init**：初始化参数，读取 config。
 - b. **Entry**：找到入口文件。
 - c. **Make**：从入口开始分析依赖，调用 **Loader** 转换文件 (TS -> JS, Less -> CSS)。
 - d. **Seal**：根据依赖关系生成 Chunk。
 - e. **Emit**：输出资源文件到 output 目录。
- **优化**:
 - **构建速度**：使用 `swc-loader` 或 `esbuild-loader` 替代 Babel；开启 `cache-loader` 持久化缓存；使用 `thread-loader` 多线程打包。
 - **打包体积**：`terser-webpack-plugin` 压缩 JS；`mini-css-extract-plugin` 抽离 CSS；分析工具 `webpack-bundle-analyzer` 找出大文件。

6. 谈谈微前端架构，为什么需要它？常用的方案有哪些？

- **为什么需要**：巨石应用 (Monolith) 难以维护；支持多团队并行开发；技术栈无关 (React/Vue 共存)；渐进式重构。
- **常用方案**:
 - a. **iframe**：最简单，隔离性最强。缺点：URL 状态不同步、通信困难、弹窗无法覆盖全局、性能较差。

- b. **qiankun (基于 single-spa)**: 目前国内最流行。HTML Entry 接入，样式隔离 (Shadow DOM/Scope CSS)，JS 沙箱 (Proxy)。
- c. **Module Federation (Webpack 5)**: 去中心化，模块共享能力强，适合组件级别的微前端，不仅仅是应用级别。
- d. **Micro App (京东)**: 基于 Web Components 实现，类似 iframe 的使用体验但没有 iframe 的缺陷。

7. 在 Agent 这种复杂状态的应用中，你会选择哪种状态管理方案？为什么？

- **选择**: 我会倾向于 **Zustand** 或 **Jotai** (原子化状态管理)，而不是 Redux。
- **原因**:
 - a. **更新频率极高**: Agent 场景全是流式数据，Redux 的 Context 机制在大频繁更新下容易导致不必要的全量重渲染。Zustand 支持 selector，可以精确控制组件更新。
 - b. **无模板代码**: Agent 逻辑复杂，不需要 Redux 那样繁琐的 Action/Reducer 模版。
 - c. **Transient Updates**: 对于打字机效果，甚至可以直接操作 DOM 或使用 `useRef` 存储临时状态，不走 React Render 流程，只在结束时同步状态。

8. 快手内部有很多跨端场景，你对跨端方案（如小程序、KwaiNative）有了解吗？

- **小程序原理：双线程模型**.
 - **渲染层**: WebView 渲染 UI。
 - **逻辑层**: JSCore 运行 JS 逻辑。
 - **通信**: 通过 Native Bridge 中转 JSON 数据。这就解释了为什么小程序 `setData` 数据量不能太大 (通信瓶颈)。
- **React Native / KwaiNative**:
 - 使用 JS 写逻辑，但 UI 渲染映射为 **原生组件 (Native Views)**，而不是 DOM。性能优于 WebView，接近原生。
- **Flutter**:
 - 自带渲染引擎 (Skia/Impeller)，不依赖系统组件，真正的一套代码画所有端。

9. 前端如何捕获并处理异常？对于白屏异常怎么监控？

- **异常捕获**:
 - JS 运行时错误: `window.onerror`。
 - Promise 没 catch: `unhandledrejection`。
 - 资源加载失败: `window.addEventListener('error', ..., true)` (捕获阶段)。

- React 组件崩溃: `Error Boundary` (`componentDidCatch`)。
- 白屏监控:
 - **Elements 检查**: 在页面加载完成 (onload) 后, 检查根节点 (如 `#root`) 内部是否有子节点, 或者关键 DOM 及其高度是否正常。
 - **采样对比**: 通过 `document.elementsFromPoint` 在屏幕中心取点, 看是否只获取到了 `html` 或 `body` 标签。

10. 手写题: 实现一个带并发限制的异步调度器 Scheduler

- **思路**: 维护一个 `queue` 队列和一个计数器 `count`。当 `count < max` 时, 出队执行, `count++`; 任务完成后 `count--` 并尝试执行下一个任务。

JavaScript

代码块

```
1  class Scheduler {
2    constructor(limit) {
3      this.limit = limit;
4      this.count = 0;
5      this.queue = [];
6    }
7
8    add(promiseCreator) {
9      return new Promise((resolve, reject) => {
10        // 将任务包装, 为了能在执行时 resolve 外部的 promise
11        const task = () => {
12          this.count++;
13          promiseCreator()
14            .then(resolve)
15            .catch(reject)
16            .finally(() => {
17              this.count--;
18              this.run(); // 任务完成, 尝试调度下一个
19            });
20          this.queue.push(task);
21          this.run();
22        };
23      });
24
25      run() {
26        if (this.count < this.limit && this.queue.length > 0) {
27          const task = this.queue.shift();
28          task();
29        }
30      }
31    }
32  }
```



```
31  }
32
33  // 使用示例const timeout = (time) => new Promise(resolve => setTimeout(resolve,
    time));
34  const scheduler = new Scheduler(2);
35  const addTask = (time, order) => {
36    scheduler.add(() => timeout(time)).then(() => console.log(order));
37  };
38
39  addTask(1000, '1');
40  addTask(500, '2');
41  addTask(300, '3');
42  addTask(400, '4');
43  // 输出顺序: 2, 3, 1, 4
```

11. 谈谈 Canvas 和 SVG 的区别，在 Agent 的工作流（Workflow）画布中你会选哪个？

• 区别：

- **SVG**：基于 DOM，矢量图，支持事件处理器，放大不失真。适合节点数量较少（<1000）、交互复杂的情况。
- **Canvas**：位图，基于像素，高性能，适合成千上万个节点。缺点是事件处理麻烦（需要自己算坐标碰撞）。

• 选型：

- 如果 Agent 工作流比较简单（几十个节点），首选 **SVG**（如使用 `React Flow` 库），开发效率高，CSS 定制样式容易。
- 如果需要展示超复杂的大规模编排（如 AI 模型层级图，上万节点），选 **Canvas**（如 `AntV X6` 或 `G6` 的 Canvas 模式）。

12. 什么是 Service Worker？它能为 Agent 应用带来什么提升？

• 定义：运行在浏览器后台的脚本，独立于网页，充当网络代理。

• Agent 场景价值：

- 离线访问**：缓存 HTML/CSS/JS shell，即使断网也能打开 APP 查看历史消息。
- 请求拦截与缓存**：缓存大型的模型配置或静态资源。
- 后台同步 (Background Sync)**：在用户网络不稳时发送的消息，可以在后台等待网络恢复后自动发送，无需用户停留在页面。

三面（架构与综合能力）

13. 如果让你从零设计一个 Agent 智能体平台的前端架构，你会考虑哪些核心模块？

这是一道系统设计题，建议画图或分层阐述：

1. 视图层 (View Layer):

- **Chat UI**: 流式渲染器、多模态消息展示（图表、Markdown、Code）。
- **Canvas Editor**: 工作流编排器（拖拽、连线）。

2. 逻辑控制层 (Controller/Store):

- **Session Manager**: 管理多会话状态、上下文切换。
- **SSE/WebSocket Handler**: 统一处理流式连接、重连、心跳、断点续传。
- **Plugin System**: 设计插件机制，允许业务方注入自定义渲染组件（例如：股票卡片、天气卡片）。

3. 数据层 (Data Layer):

- **IndexedDB**: 存储海量聊天记录（LocalStorage 不够用）。
- **Vector Cache**: 如果前端做轻量级 RAG，可能需要缓存部分向量数据。

4. 基础设施 (Infra):

- **BFF (Backend for Frontend)**: 处理跨域、鉴权、流转换。
- **Observability**: 全链路埋点（监控 Token 耗时、首字时间）。

14. 在 Agent 项目中，前端如何参与 Prompt Engineering（提示词工程）的优化？

前端不仅仅是展示，还能辅助 Prompt 调优：

1. 可视化 Prompt 编排: 开发“结构化 Prompt 编辑器”，将 System Prompt、Few-Shot Examples、User Input 模块化，方便非技术人员（PM/运营）调试。

2. 上下文注入 (Context Injection): 前端收集用户环境信息（当前页面 URL、选中的文本、操作系统、时区），自动拼接进 Prompt，让 AI 回答更精准。

3. A/B Testing 平台: 前端配合后端实现 Prompt 的灰度发布。同一问题，A 组用 Prompt V1，B 组用 V2，通过前端埋点（点赞/点踩/留存时长）评估 Prompt 质量。